

Python - Arbitrary or Variable-length Arguments

Arbitrary Arguments (*args)

You may want to define a **function** that is able to accept **arbitrary** or **variable number of arguments**. Moreover, the arbitrary number of arguments might be **positional** or **keyword arguments**.

- An argument prefixed with a single asterisk * for arbitrary positional arguments.
- An argument prefixed with two asterisks ** for arbitrary keyword arguments.

Arbitrary Arguments Example

Given below is an example of arbitrary or variable length positional arguments –

```
# sum of numbers
def add(*args):
    s=0
    for x in args:
        s=s+x
    return s
result = add(10,20,30,40)
print (result)
```

```
result = add(1,2,3)
print (result)
```

The **args** variable prefixed with "*" stores all the values passed to it. Here, args becomes a tuple. We can run a **loop** over its items to add the numbers.

It will produce the following **output** –

```
100
6
```

Required Arguments With Arbitrary Arguments

It is also possible to have a function with some required arguments before the sequence of **variable** number of values.

Example

The following example has **avg()** function. Assume that a student can take any number of tests. First test is mandatory. He can take as many tests as he likes to better his score. The function calculates the average of marks in first test and his maximum score in the rest of tests.

The function has two arguments, first is the required argument and second to hold any number of values.

```
#avg of first test and best of following tests
def avg(first, *rest):
    second=max(rest)
    return (first+second)/2

result=avg(40,30,50,25)
print (result)
```

Following call to avg() function passes first value to the required argument first, and the remaining values to a tuple named rest. We then find the maximum and use it to calculate the average.

It will produce the following **output** –

45.0

Arbitrary Keyword Arguments (**kwargs)

If a variable in the argument list has two asterisks prefixed to it, the function can accept arbitrary number of keyword arguments. The variable becomes a **dictionary** of keyword:value pairs.

Example

The following code is an example of a function with arbitrary keyword arguments. The `addr()` function has an argument `**kwargs` which is able to accept any number of address elements like name, city, phno, pin, etc. Inside the function `kwargs` dictionary of `kw:value` pairs is traversed using `items()` method.

```
def addr(**kwargs):  
    for k,v in kwargs.items():  
        print ("{}:{}".format(k,v))  
  
print ("pass two keyword args")  
addr(Name="John", City="Mumbai")  
print ("pass four keyword args")  
  
# pass four keyword args  
addr(Name="Raam", City="Mumbai", ph_no="9123134567", PIN="400001")
```

It will produce the following **output** –

```
pass two keyword args  
Name:John  
City:Mumbai  
pass four keyword args  
Name:Raam  
City:Mumbai  
ph_no:9123134567  
PIN:400001
```

Multiple Arguments With Arbitrary Keyword Arguments

If the function uses mixed types of arguments, the arbitrary keyword arguments should be after positional, keyword and arbitrary positional arguments in the argument list.

Example

Imagine a case where science and maths are mandatory subjects, in addition to which student may choose any number of elective subjects.

The following code defines a **percent()** function where marks in science and marks are stored in required arguments, and the marks in variable number of elective subjects in ****optional** argument.

```
def percent(math, sci, **optional):
    print ("maths:", math)
    print ("sci:", sci)
    s=math+sci
    for k,v in optional.items():
        print ("{}:{}".format(k,v))
        s=s+v
    return s/(len(optional)+2)

result=percent(math=80, sci=75, Eng=70, Hist=65, Geo=72)
print ("percentage:", result)
```

It will produce the following **output** –

```
maths: 80
sci: 75
Eng:70
Hist:65
Geo:72
percentage: 72.4
```

[Python Tutorial](#)
[Java Tutorial](#)
[C++ Tutorial](#)
[C Programming Tutorial](#)
[C# Tutorial](#)
[PHP Tutorial](#)
[R Tutorial](#)
[HTML Tutorial](#)
[CSS Tutorial](#)
[JavaScript Tutorial](#)
[SQL Tutorial](#)

TRENDING TECHNOLOGIES

[Cloud Computing Tutorial](#)
[Amazon Web Services Tutorial](#)
[Microsoft Azure Tutorial](#)
[Git Tutorial](#)
[Ethical Hacking Tutorial](#)
[Docker Tutorial](#)
[Kubernetes Tutorial](#)
[DSA Tutorial](#)
[Spring Boot Tutorial](#)
[SDLC Tutorial](#)
[Unix Tutorial](#)

CERTIFICATIONS

[Business Analytics Certification](#)
[Java & Spring Boot Advanced Certification](#)



Chapters ▾

Categories ≡

[Advanced Certification In Business Analytics](#)
[Artificial Intelligence And Machine Learning](#)
[DevOps Certification](#)
[Game Development Certification](#)
[Front-End Developer Certification](#)
[AWS Certification Training](#)

COMPILERS & EDITORS

[Online Java Compiler](#)

[Online Python Compiler](#)

[Online Go Compiler](#)

[Online C Compiler](#)

[Online C++ Compiler](#)

[Online C# Compiler](#)

[Online PHP Compiler](#)

[Online MATLAB Compiler](#)

[Online Bash Terminal](#)

[Online SQL Compiler](#)

[Online Html Editor](#)

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.